

SC05 | University Timetable Optimization Engine

Difficulty: Advanced capstone **Core techniques:** Genetic Algorithm **Team:** 3-4 students

1. What you are building

Create a working decision-support product that converts **courses, rooms, faculty, cohorts and constraints** into **conflict-free timetable and quality score**. The final system must be runnable by a user, explain its result visually, and demonstrate why Soft Computing is more suitable than a rigid rule-only approach.

2. Why this is a Soft Computing capstone

- The problem contains uncertainty, non-linear behaviour, competing goals, or incomplete information.
- You must expose the important algorithmic elements rather than only calling a library function.
- A baseline is compulsory: random or greedy timetable. Your evidence must show what the Soft Computing solution improves.

3. Product and system architecture

Pipeline: data/scenario collection → validation and preprocessing → baseline → Genetic Algorithm model → decision/optimised output → evaluation dashboard.

The application may use Streamlit, a clear notebook interface, GUI, or command-line interface with meaningful visualisation. It must accept inputs, run the method, return a result, and show at least one explanation, plot, table, schedule, route, or membership/fitness view.

4. Data and experimental strategy

Required source: 10,000+ documented timetable scenario variations. Document dataset name, URL, licence, original size, features/targets, all preprocessing, and any scenario-generation assumptions. The minimum is 10,000 records or realistic test scenarios. Do not invent a citation: state clearly what is real and what is simulated.

5. Milestone 1 - Working Product V1

Implement chromosome, fitness, selection, crossover, mutation, repair and a timetable viewer.

M1 deliverables and acceptance test:

- Repository created early, with README, requirements, source/app folders, data citation, and continuous commits.
- At least 10,000 records/scenarios documented and a reproducible preprocessing or generation step.
- One core Soft Computing algorithm working, plus the stated baseline.
- A user can demonstrate input → algorithmic processing → output. Documentation-only work is not accepted.
- At least two initial visuals: for example loss/error curve, membership plot, fitness curve, route/schedule, predicted-vs-actual, or baseline comparison.
- Interim report: problem, data, architecture, implementation, MVP screenshots, initial results, limitations and next steps.

6. Milestone 2 - Enhanced Product V2

Add multi-objective constraints and parameter studies; compare GA and heuristic quality at scale.

M2 deliverables and acceptance test:

- Complete the advanced/hybrid component and make the final application cleanly runnable from README instructions.
- Run parameter experiments rather than a single best run. Explain settings, results, and decisions.
- Compare baseline, M1 approach, and advanced M2 approach where applicable, using hard violations, soft penalty, runtime and fitness.
- Deliberately test edge cases: noise, extreme values, contradictory inputs, unseen cases, or difficult constraints.
- Publish final plots/tables, report, ~5-minute demo video, presentation material, and team contribution details.

7. Required repository evidence

- README with team, problem, architecture, dataset citation, setup/run commands, results and limitations.
- src/ for reusable algorithms; app/ for runnable product; results/ for labelled outputs; report/ for reports; data/README.md for source information.
- Meaningful commits throughout the semester, such as “Implement fuzzy rules”, “Add GA mutation”, or “Evaluate baseline”. A final ZIP/code dump does not earn full Git marks.

8. Evaluation evidence (50 marks)

GitHub - 10 marks Structure, code quality, meaningful commits, and visible contribution.	Report - 10 marks Problem, data citation, methodology, implementation, experiments, and results.
Video - 10 marks Problem, concept explanation, product demo, comparison, and clarity.	Viva & Presentation - 20 marks Technique understanding, own code, results, technical answers, and coordination.

Final reminder

This is an engineering product capstone. A strong submission makes it easy for a faculty member to run the project, inspect the evidence, compare it with the baseline, and ask each student about the part they built.